Firebase Authentication

Sinh viên: Trần Viết Thắng
Sinh viên: Trần Duy Nguyên

Khoa điện tử viễn thông
Đại học bách khoa
Đại học Đà Nẵng

Ngày 13 tháng 11 năm 2025

Phân công công việc

Họ và tên	Nhiệm vụ	Phần trăm
Trần Duy Nguyên	- Quản lí người dùng và các phương thức bảo mật. - Xây dựng giao diện app.	50%
Trần Viết Thắng	- Các cách thức đăng nhập lên Firebase bằng Email, Google và Facebook. - Xây dựng giao diện ứng dụng.	50%

Outline

1. Overview
2. Email/Password login
3. Social Login
   - Google Login
   - Facebook Login
4. User Management
5. Security
6. Tài liệu tham khảo

Table of Contents

1. Overview
2. Email/Password login
3. Social Login
   - Google Login
   - Facebook Login
4. User Management
5. Security
6. Tài liệu tham khảo

Sơ lược về firebase

- Khái niệm:
  - Backend-as-a-Service (BaaS) - phát triển bởi google.
  - Cho phép người dùng xây dựng ứng dụng mà không cần quản lý server.
  - Cung cấp nhiều dịch vụ Authentication, Database, Storage, Hosting.
- Lợi ích:
  - → Set up nhanh, không cần quan tâm đến backend API.
  - → Dịch vụ miễn phí, phù hợp cho các dự án nhỏ.
  - → Cho phép đăng tải dữ liệu thời gian thực.
  - → Tài liệu rõ ràng, dễ dùng.

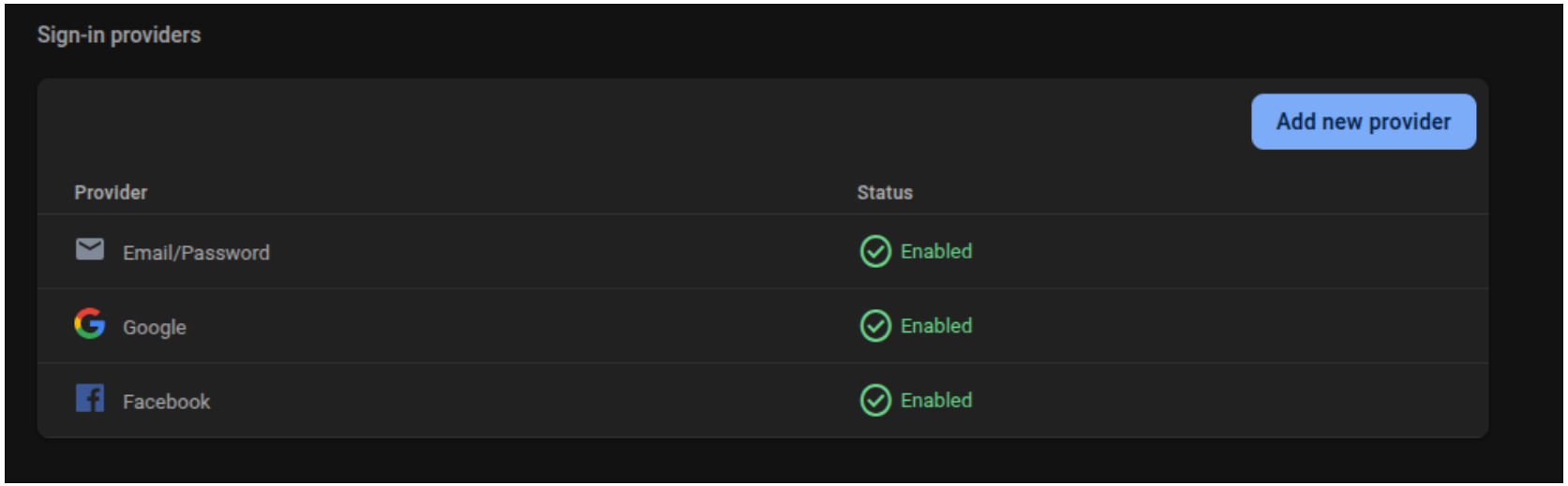
Firebase Authentication

1. Thông tin xác thực (gmail, password hoặc mã xác thực từ các nhà cung cấp).
2. Các thông tin xác thực gửi đến firebase SDK.
3. Xác thực thông tin và phản hồi lại.
4. Thành công : trả về thông tin cơ bản của người dùng và token xác thực.

Table of Contents

1. Overview
2. Email/Password login
3. Social Login
   - Google Login
   - Facebook Login
4. User Management
5. Security
6. Tài liệu tham khảo

Bật quyền cho các nhà cung cấp



1. Ta nhấn vào nút "Add new provider" ở mục signed-in provider.
2. Chọn nhà cung cấp mong muốn và nhấn vào nút "Enable".
3. Thành công sẽ cho ta thấy thanh "Status" như trên hình.
4. Ứng với mỗi nhà cung cấp, ta cần làm theo các bước trong Firebase để có thể kết nối được với Firebase.

Đăng nhập và đăng kí bằng mail

- Thêm các gói cần thiết:
  1. Khai báo trong pubspec.yaml :

```
  firebase_core: ^3.15.2
  firebase_auth: ^5.7.0

```

- Thực hiện các bước kết nối với Firebase.
- Chú ý: Các bước này cần thực hiện theo hướng dẫn để có kết nối với dịch vụ của Firebase.

Đăng nhập và đăng kí bằng email

- Xử lí đăng nhập:

```
final userCredential = await FirebaseAuth.instance.signInWithEmailAndPassword(
    email: _emailController.text.trim(),
    password: _passwordController.text,
    );

```

- Sử dụng FirebaseAuth.instance.signInWithEmailAndPassword.
- _emailController.text.trim() : Lấy nội dung người dùng nhập vào ô email, rồi loại bỏ khoảng trắng ở đầu/cuối.

→ Firebase trả về UserCredential → Giúp cho công việc xác thực email người dùng.

- Xử lí đăng kí:

```
final userCredential = await FirebaseAuth.instance.
     createUserWithEmailAndPassword(
          email: _emailController.text.trim(),
          password: _passwordController.text,
);
```

Đăng nhập và đăng kí bằng mail

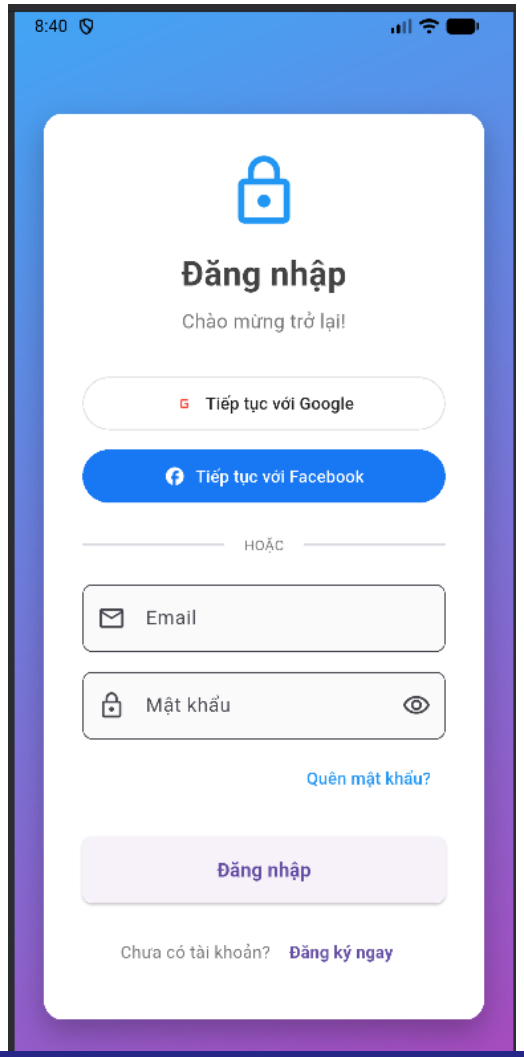
- Sử dụng try cacth cho 2 hàm đăng nhập và đăng kí

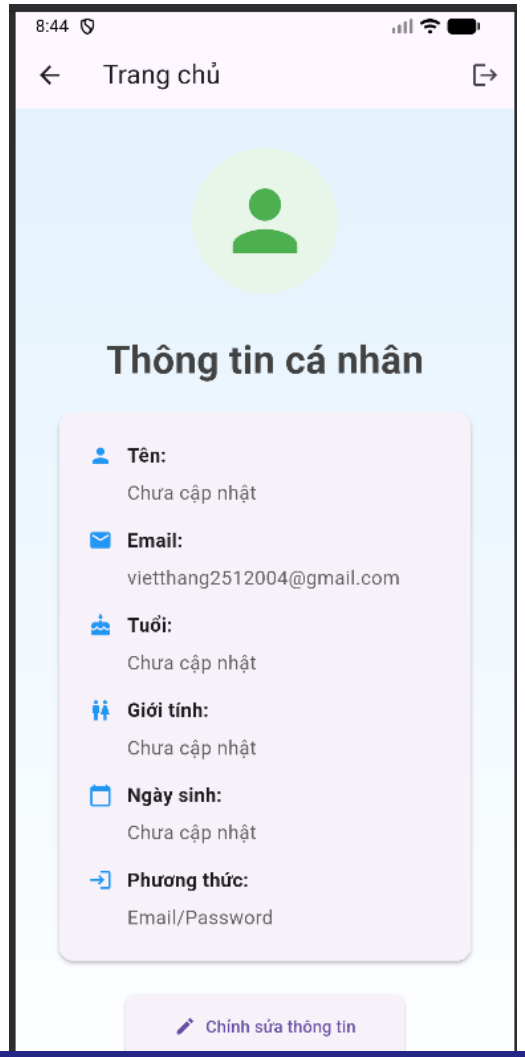
```
try{
 //
} on FirebaseAuthException catch (e) {
  String message = 'Da xay ra loi';
  if (e.code == 'user-not-found') message = 'Khong tim thay tai khoan';
  else if (e.code == 'wrong-password') message = 'sai mat khau';
  // ...
  _showError(message);}

```

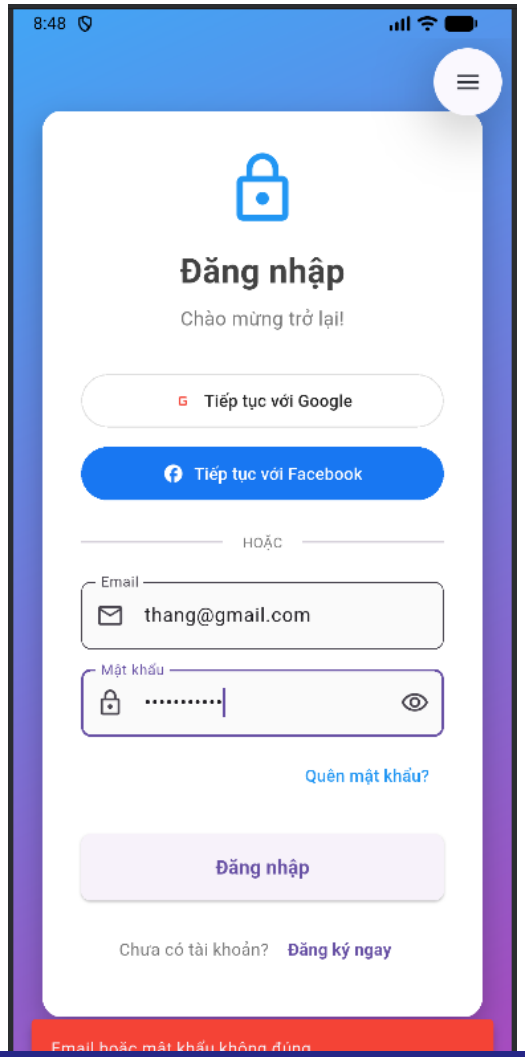
- FirebaseAuthException trả về các loại lỗi:

→ user-not-found: Email chưa được đăng ký.
→ wrong-password: Mật khẩu sai.
→ email-already-in-use: Email đã có tài khoản.
→ weak-password: Mật khẩu quá yếu.
→ invalid-email: Email sai định dạng.
→ invalid-credential: Thông tin không hợp lệ.

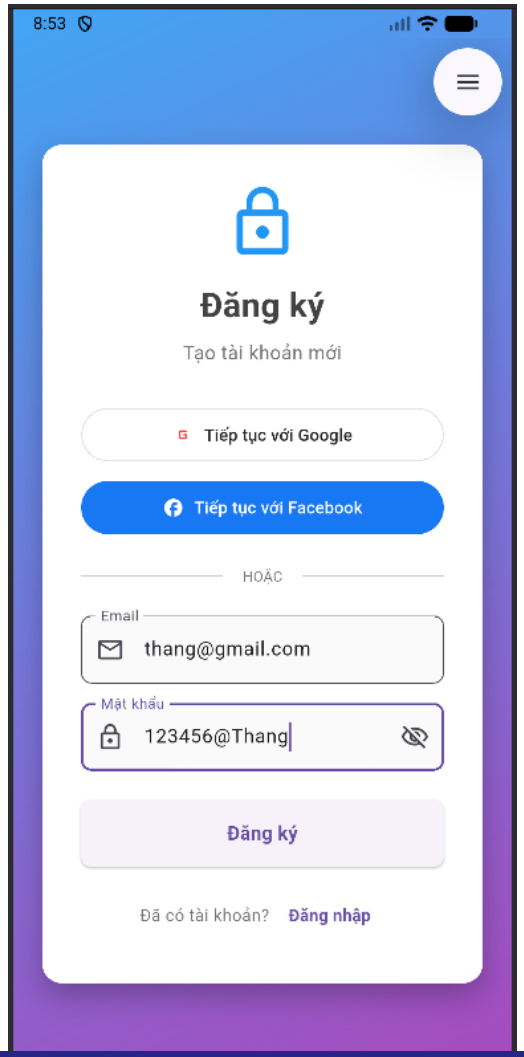
Demo đăng nhập







Demo đăng kí



Identifier	Providers	Created ↓	Signed In	User UID
thang@gmail.com		Nov 11, 2025	Nov 11, 2025	3AcNhuM27zTzqqrgifOrZfxbH...
vietthang2512004@gm...		Nov 10, 2025	Nov 11, 2025	3G31o1gR4dPlyud3OxiNqPDf...
duynguyen25092004@...		Nov 10, 2025	Nov 10, 2025	d2IFYjSveNbkx6bJ34rzTEcmE...
hellosigma@gmail.com		Nov 10, 2025	Nov 10, 2025	mvFzrLedq2OeoBZaAkJ2BfGe...
trannguyendut@gmail.c...		Nov 10, 2025	Nov 10, 2025	tTsGNtoJjVbFC8Rii1kie6SHiyr1
mhang0181@gmail.com		Nov 10, 2025	Nov 10, 2025	GvJZR6g21nVpUyAUCYfiDoxY...
minhhangdo2004@gm...		Nov 10, 2025	Nov 10, 2025	fcZUPeYlE1d6sLgf6Y3G9KPT...
thangtran2512004@gm...		Nov 9, 2025	Nov 10, 2025	k8sapje7e2YB7xXHOcoC0j8m...

Table of Contents

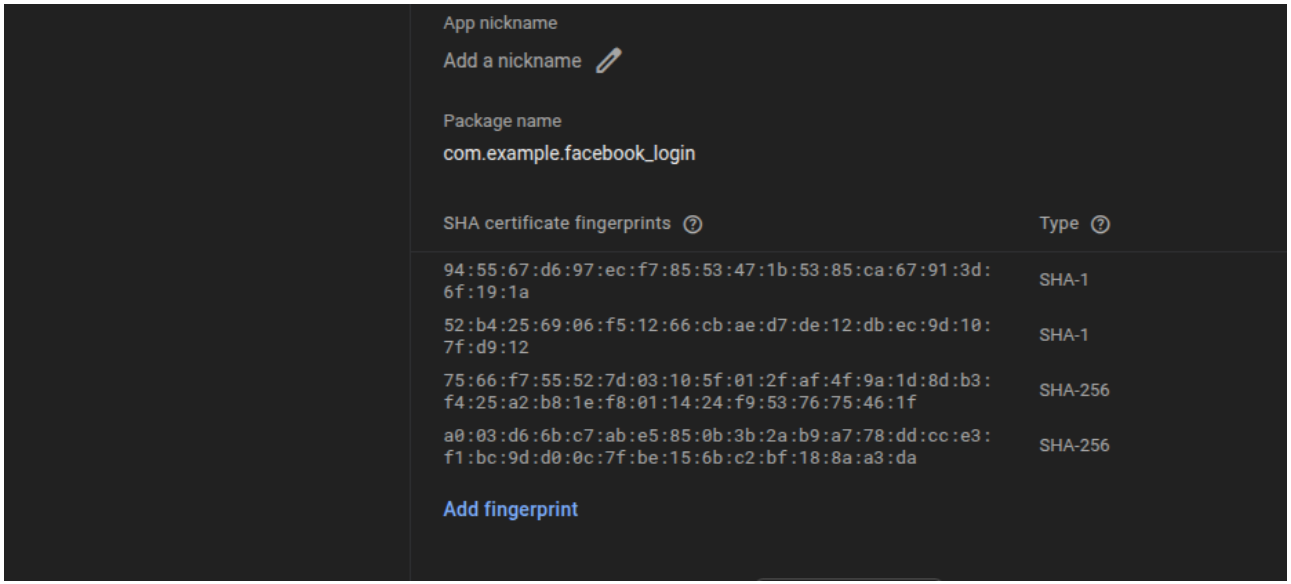
1. Overview
2. Email/Password login
3. Social Login
   - Google Login
   - Facebook Login
4. User Management
5. Security
6. Tài liệu tham khảo

Cấu hình trên Firebase

- Để có thể kết nối với phương thức này ta cần 2 mã SHA1 và SHA256 của máy đang chạy ứng dụng.
- Sử dụng các câu lệnh như :

```
keytool -list -v \-alias androiddebugkey -keystore ~/.android/debug.keystore

```

- Thêm vào mục setting của dự án Firebase:



Google login

```
Future<void> _signInWithGoogle() async {
    //... UI handling ... //
try {
 final GoogleSignIn googleSignIn = GoogleSignIn();
 final GoogleSignInAccount? googleUser = await googleSignIn.signIn();
  if (googleUser == null) {
    setState(() => _isLoading = false);
    return;
  }
  final GoogleSignInAuthentication googleAuth = await googleUser.authentication;
  final credential = GoogleAuthProvider.credential(
        accessToken: googleAuth.accessToken,
        idToken: googleAuth.idToken,
);
await FirebaseAuth.instance.signInWithCredential(credential);
    } on FirebaseAuthException catch (e) {
      if (e.code == 'account-exists-with-different-credential') {
        _pendingCredential = e.credential;
        await _handleAccountConflict(e, _pendingCredential);
      } else {
        _showError('Dang nhap google that bai: ${e.toString()}');
      }
    } catch (e) {
      _showError('Dang nhap google that bai: ${e.toString()}');
    } finally { // UI handling //}
  }
```

Google login

- Đăng nhập vào Google:

```
final GoogleSignIn googleSignIn = GoogleSignIn();
final GoogleSignInAccount? googleUser = await googleSignIn.signIn();
```

- GoogleSignIn : Tạo một instance của class GoogleSignIn từ package.
- googleSignIn.signIn(): trả về một đối tượng của lớp GoogleSignInAccount. Nếu người dùng hủy đăng nhập thì trả về null.

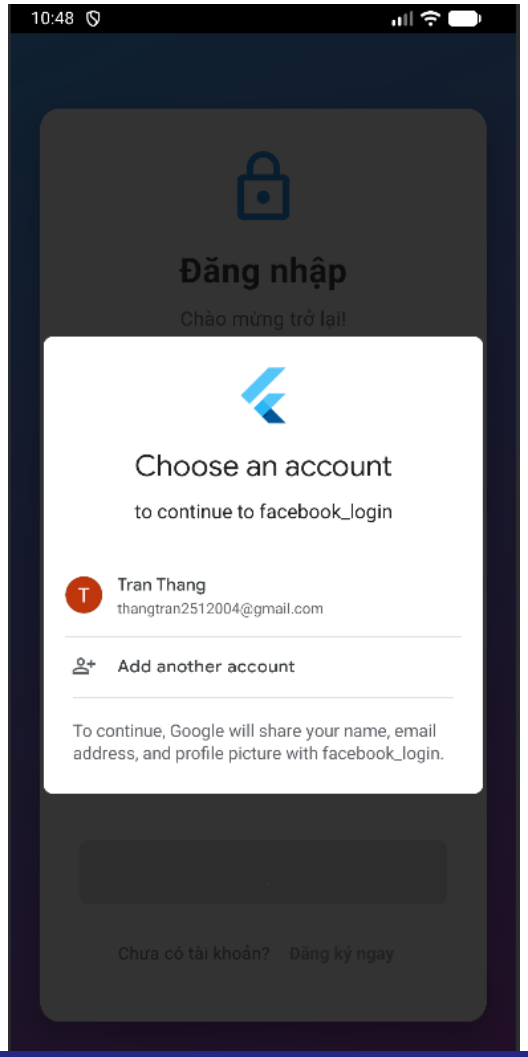
Google login

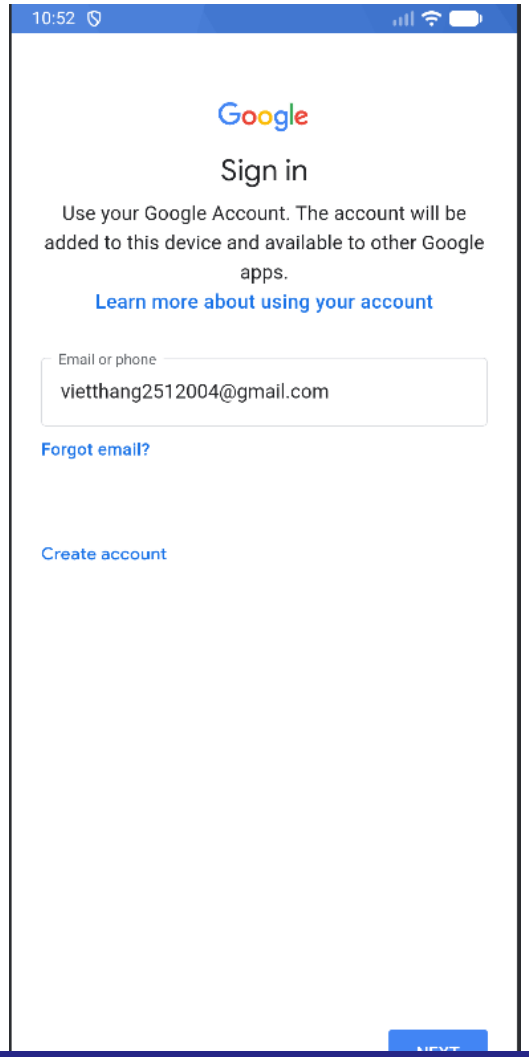
- Lấy token xác thực và tạo Firebase Credential.

```
final GoogleSignInAuthentication googleAuth = await googleUser.authentication;
final credential = GoogleAuthProvider.credential(
 accessToken: googleAuth.accessToken,
 idToken: googleAuth.idToken,
);
await FirebaseAuth.instance.signInWithCredential(credential);
```

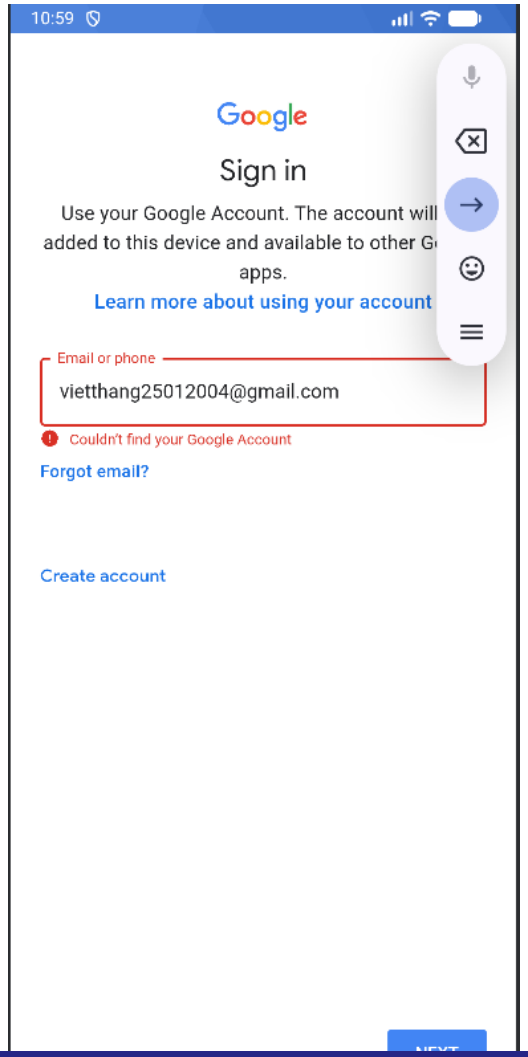
- Đăng nhập thành công, Google sẽ cấp 2 loại token:
  - Access Token : truy cập vào Google API.
  - Id Token : Xác thực người dùng.
- Phương thức .credential() chuyển đổi token thành Firebase credential.
- Phương thức .signInWithCredential() với credential vừa tạo để truy cập vào Firebase.

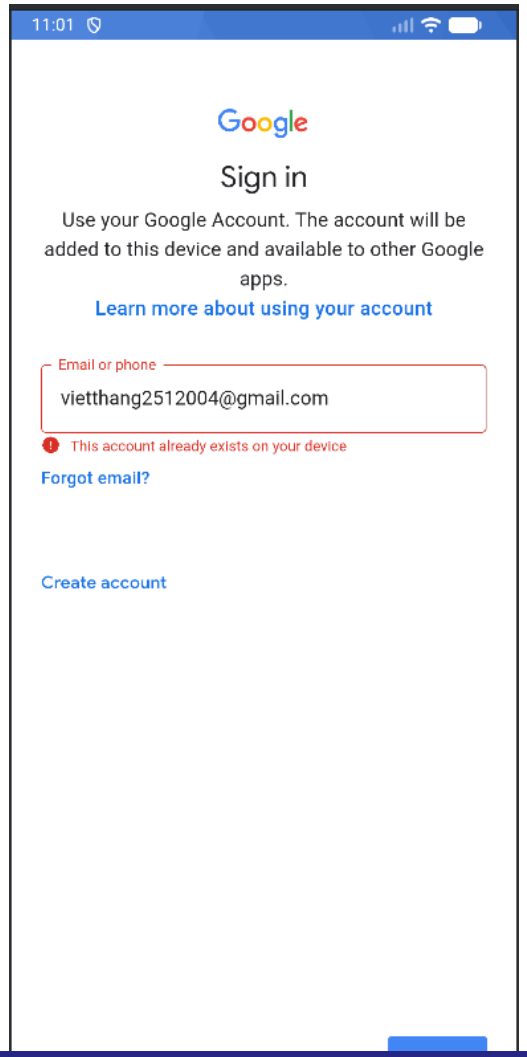
Demo đăng nhập bằng google

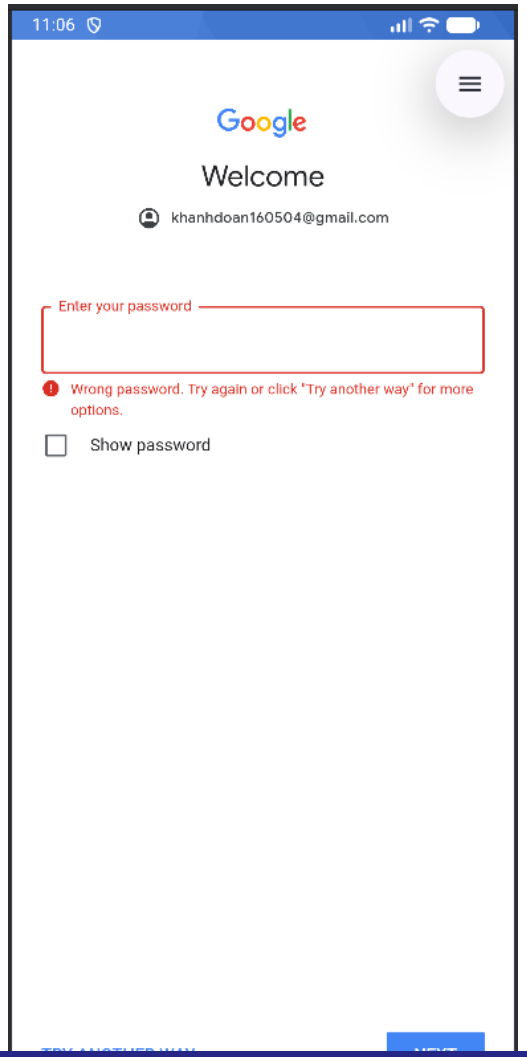




Demo một số lỗi xảy ra







Facebook Login

```
Future<void> _signInWithFacebook() async {
   // UI Handling ....
try {
  final LoginResult result = await FacebookAuth.instance.login();
  if (result.status == LoginStatus.success) {
   final OAuthCredential credential = FacebookAuthProvider.credential(result.
     accessToken!.token);
   await FirebaseAuth.instance.signInWithCredential(credential);
   } else if (result.status == LoginStatus.cancelled) {
    _showError('Dang nhap Facebook bi huy');
   } else {
    _showError('Dang nhap Facebook that bai');
    }
} on FirebaseAuthException catch (e) {
  if (e.code == 'account-exists-with-different-credential') {
     _pendingCredential = e.credential;
     await _handleAccountConflict(e, _pendingCredential);
   } else {
     _showError('Loi Facebook: ${e.toString()}');
    }
   } catch (e) {
      _showError('Loi Facebook: ${e.toString()}');
   } finally { // UI Handling ...
   }
  }
```

Facebook login

- Đăng nhập vào Facebook:

```
final LoginResult result = await FacebookAuth.instance.login();
```

- Thực hiện đăng nhập trả về đối tượng kiểu - LoginResult.
  - status : sucess, failed, cancelled, operationInProgress.
  - accessToken : token trả về nếu khởi tạo thành công.
  - message : lỗi.
- Ta dựa vào 3 thuộc tính trên để xây dựng:

```
if (result.status == LoginStatus.success) {
// .. process for success .
}
else {
// .. process for failure.
}
```

Facebook login

- Xử lí khi đăng nhập thành công:

```
if (result.status == LoginStatus.success) {
  final OAuthCredential credential = FacebookAuthProvider.credential(result.
     accessToken!.token);
  await FirebaseAuth.instance.signInWithCredential(credential);
}
```

- FacebookAuthProvider.credential() :
  - Chuyển đổi thành OAuthCredential cho Firebase.
- FirebaseAuth.instance.signInWithCredential():
  - Gửi credential lên Firebase.
  - Firebase verify với Facebook server.
  - Tạo/cập nhật user trong Firebase Auth.

Demo đăng nhập bằng Facebook

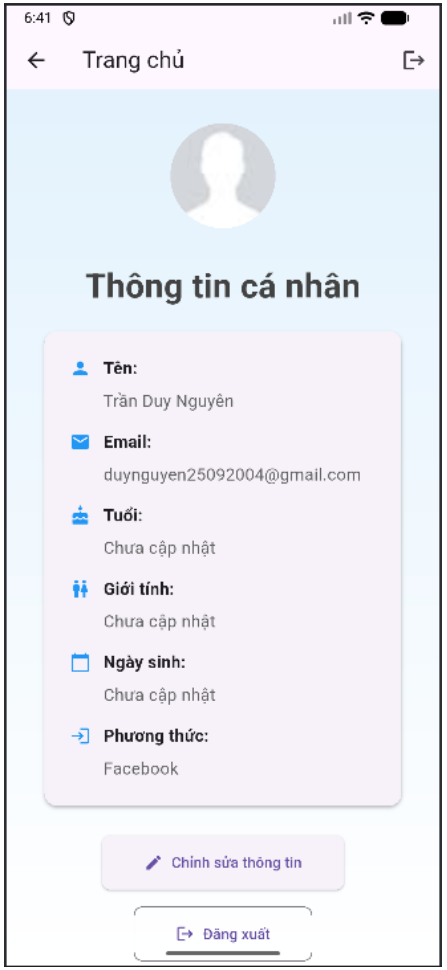


Table of Contents

1. Overview
2. Email/Password login
3. Social Login
   - Google Login
   - Facebook Login
4. User Management
5. Security
6. Tài liệu tham khảo

Tại sao lại cần quản lý người dùng ?

- Hầu hết các ứng dụng cần biết danh tính của người dùng để:
  - Lưu dữ liệu người dùng an toàn trên đám mây.
  - Cung cấp trải nghiệm cá nhân hoá nhất quán trên mọi thiết bị.
- Việc quản lý người dùng là phần không thể thiếu khi xây dựng ứng dụng, nhưng thường phức tạp và tiềm ẩn rủi ro bảo mật.
- Với **Firebase Authentication**, nhà phát triển không cần tự xây dựng hệ thống xác thực và phân quyền từ đầu.
- Firebase cung cấp các thư viện và dịch vụ xác thực an toàn, giúp quản lý người dùng dễ dàng và giảm thiểu lỗi bảo mật.

Các phương thức quản lý người dùng

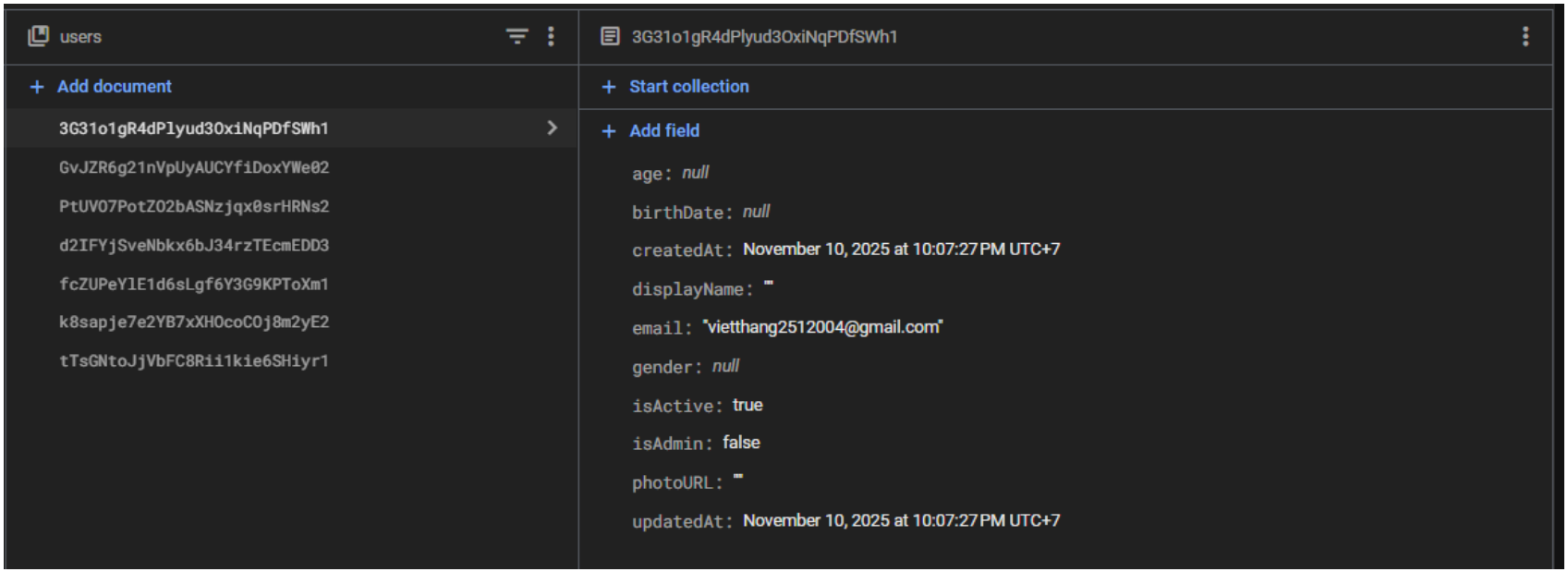
Quản lý từ phía Người dùng (User)

- Đăng ký tài khoản (Email/Password hoặc Google, Facebook).
- Hiển thị/cập nhật thông tin cá nhân.
- Gửi yêu cầu xác thực qua Email (xác minh tài khoản, đặt lại mật khẩu).

Quản lý từ phía Quản trị viên (Admin)

- Quản lý người dùng: xem danh sách, tìm kiếm, kích hoạt/vô hiệu hóa tài khoản.
- Xem chi tiết người dùng trong Firestore (email, tuổi, giới tính, trạng thái, ngày tạo,...).
- Xóa tài khoản người dùng.

Cấu trúc cơ sở dữ liệu trong Firebase



- Trong Firestore Database, mỗi user (tương ứng với một tài khoản Email) sẽ được phân biệt bằng UID (User ID) duy nhất.
- Mỗi UID chứa dữ liệu bao gồm các **trường (fields)** và giá trị tương ứng của chúng, được tổ chức dưới dạng key-value.

Hiển thị thông tin từ Firestore Database

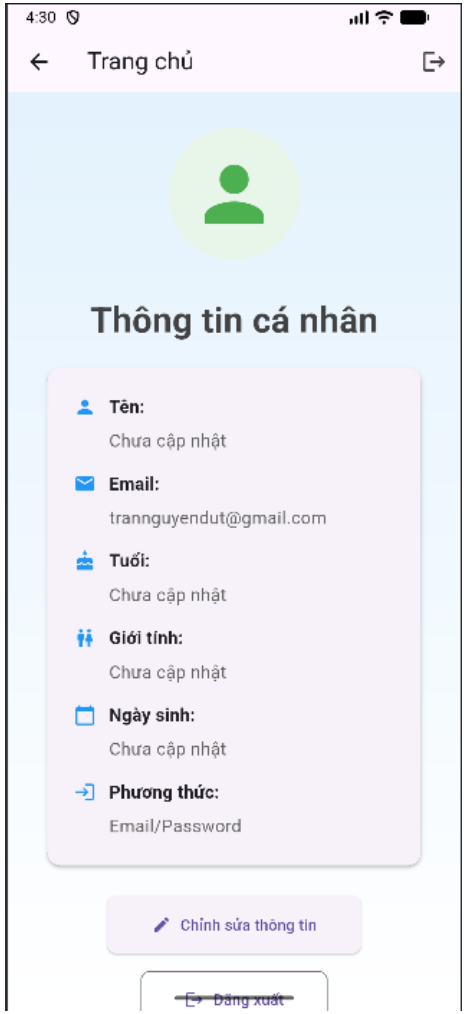
- Thêm các gói cần thiết trong `pubspec.yaml`:

```
dependencies:
  flutter:
    sdk: flutter
  firebase_auth: ^5.7.0
  cloud_firestore: ^5.6.0

```

- Các cách lấy thông tin người dùng hiện tại:
  - Qua các luồng (streams): `authStateChanges()`, `idTokenChanges()`, `userChanges()`
  - Qua đối tượng trả về từ phương thức đăng nhập: `UserCredential.user`
  - Qua thuộc tính hiện tại của FirebaseAuth: `FirebaseAuth.instance.currentUser`
  - Thông qua Firestore Database: _firestore.collection('users').doc(uid).get(), _firestore.collection('users').doc(uid).snapshots()

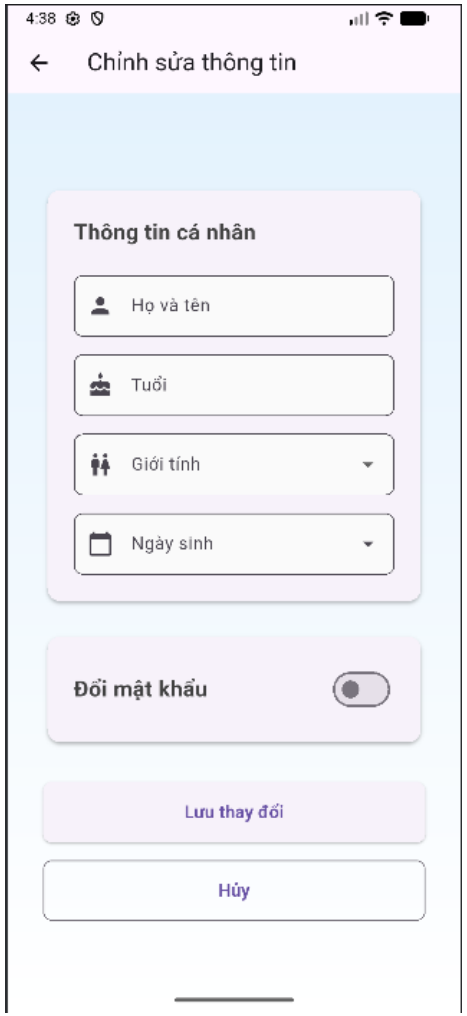
Hiển thị thông tin từ Firetore Database (Con't)

```
Future<void> _initializeUserData() async {
  setState(() => _isLoading = true);
  final user = FirebaseAuth.instance.currentUser;
  if (user != null) {
    try {
     final docRef = _firestore.collection('users').doc(
     user.uid);
     final doc = await docRef.get();
     if (!doc.exists) { //...
    } else {
      print('User document exists');
      final data = doc.data();
      setState(() {
      _userData = data;
      _isAdmin = data?['isAdmin'] == true;
      _isLoading = false;
    });
    }
    } catch (e) { //...
    }
    } else {
      setState(() => _isLoading = false);
    }
  }
```

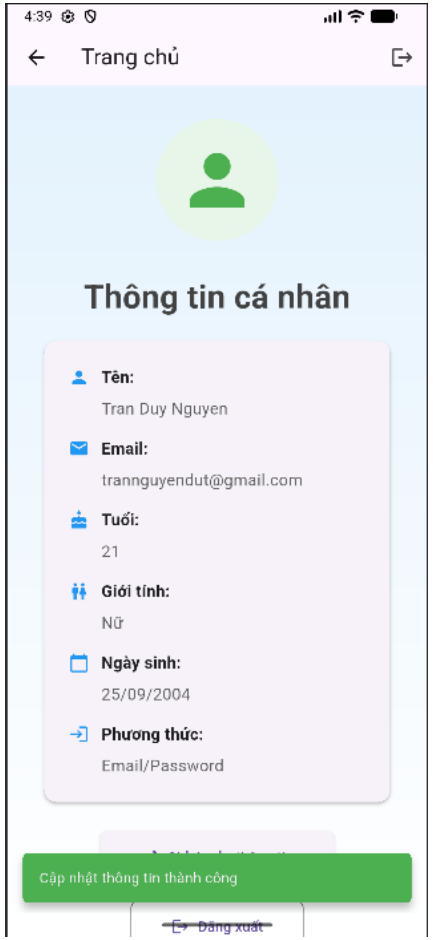


Cập nhật thông tin lên Firestore Database

```
Future<void> _saveChanges() async {
  setState(() => _isLoading = true);
  try {
    final user = FirebaseAuth.instance.currentUser;
    final updateData = {
     'displayName': _nameController.text.trim(),
     'age': _ageController.text.trim().isNotEmpty
          ? int.tryParse(_ageController.text.trim())
          : null,
     'gender': _selectedGender,
     'birthDate': _birthDateController.text.trim().
     isNotEmpty
          ? _birthDateController.text.trim()
          : null,
     'updatedAt': FieldValue.serverTimestamp(),
    };
    await _firestore.collection('users').doc(user.uid).
     update(updateData);
  } catch (e) { //...
  }
}
```



Cập nhật thông tin lên Firestore Database (Con't)

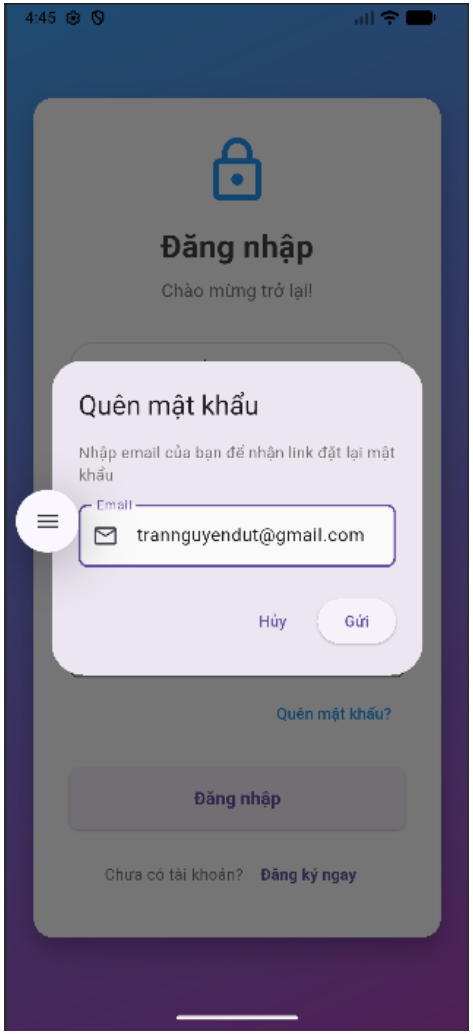


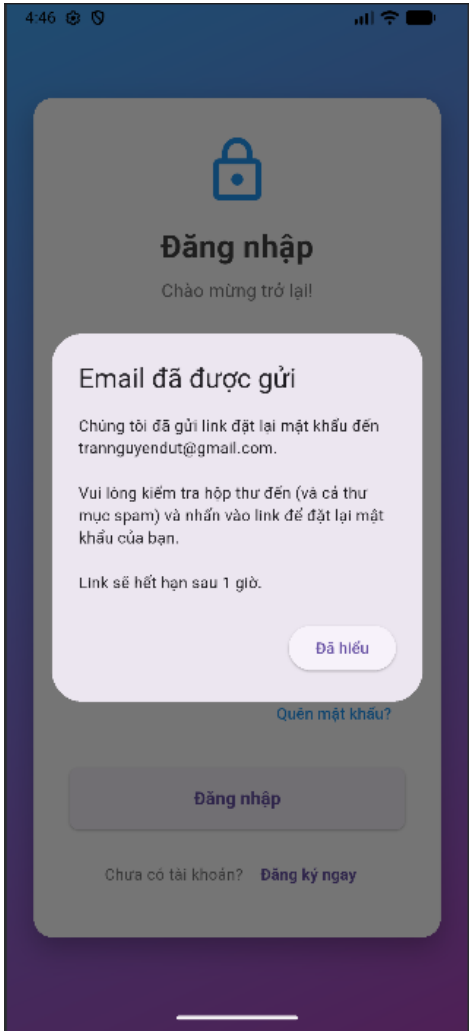
Quên mật khẩu

```
Future<void> _showForgotPasswordDialog() async {
  final emailController = TextEditingController();
  final formKey = GlobalKey<FormState>();
  final result = await showDialog<bool>(
    context: context,
    builder: (context) => AlertDialog(
      title: Text('Quen mat khau'),
    ),
    //...
  );

  if (result == true) {
    await _sendPasswordResetEmail(emailController.text.trim());
  }
  // ...
}
```

```
Future<void> _sendPasswordResetEmail(String email) async {
  try {
    await FirebaseAuth.instance.sendPasswordResetEmail(email: email);
    _showDialog(
      title: 'Email da duoc gui',
      // ...
    );
  } on FirebaseAuthException catch (e) {
      //...
  } catch (e) { //...
  }
}
```

Quên mật khẩu (Con't)





Quên mật khẩu (Con't)

Reset your password for project-474081122253 Spam ×

noreply@fir-auth-2626d.firebaseapp.com
to me

4:46 PM (1 minute ago)

Why is this message in spam? This message was marked as spam because previous messages from firebaseapp.com were marked as spam.

Report not spam

Hello,

Follow this link to reset your project-474081122253 password for your trannguyendut@gmail.com account.

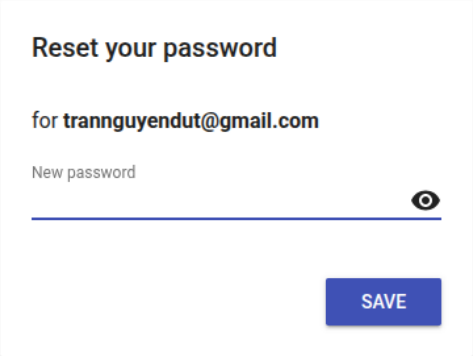
https://fir-auth-2626d.firebaseapp.com/__/auth/action?mode=resetPassword&oobCode=x8FtRwbQBhOEPWp75Ame_a5pFtu159HG1MPC7Q6HoaoAAAGack-Hbg&apiKey=AIzaSyC425d0j7CGZpoiLcev25LjTXixvS6QBLM&lang=en

If you didn't ask to reset your password, you can ignore this email.

Thanks,

Your project-474081122253 team

Reply | Forward

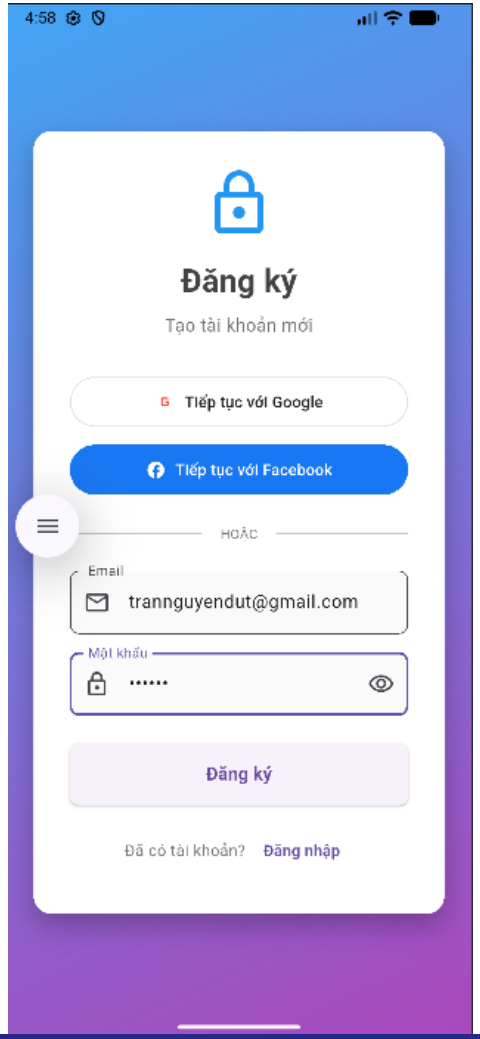


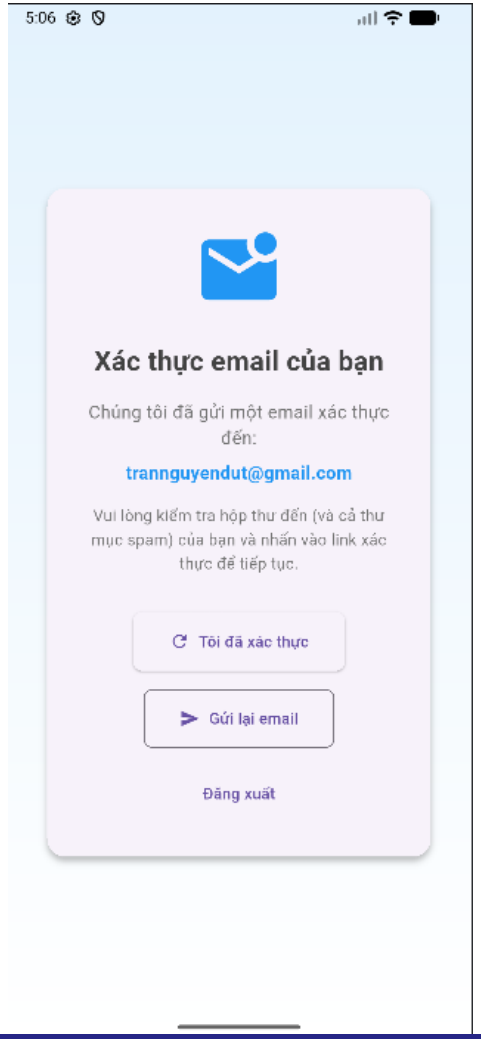
Xác minh Email

```
final userCredential = await
      FirebaseAuth.instance.
      createUserWithEmailAndPassword(
  email: _emailController.text.trim(),
  password: _passwordController.text,
);

if (userCredential.user != null) {
  await userCredential.user!.
      sendEmailVerification();
  setState(() {
    _isWaitingForVerification = true;
    _isLoading = false;
  });
}
```

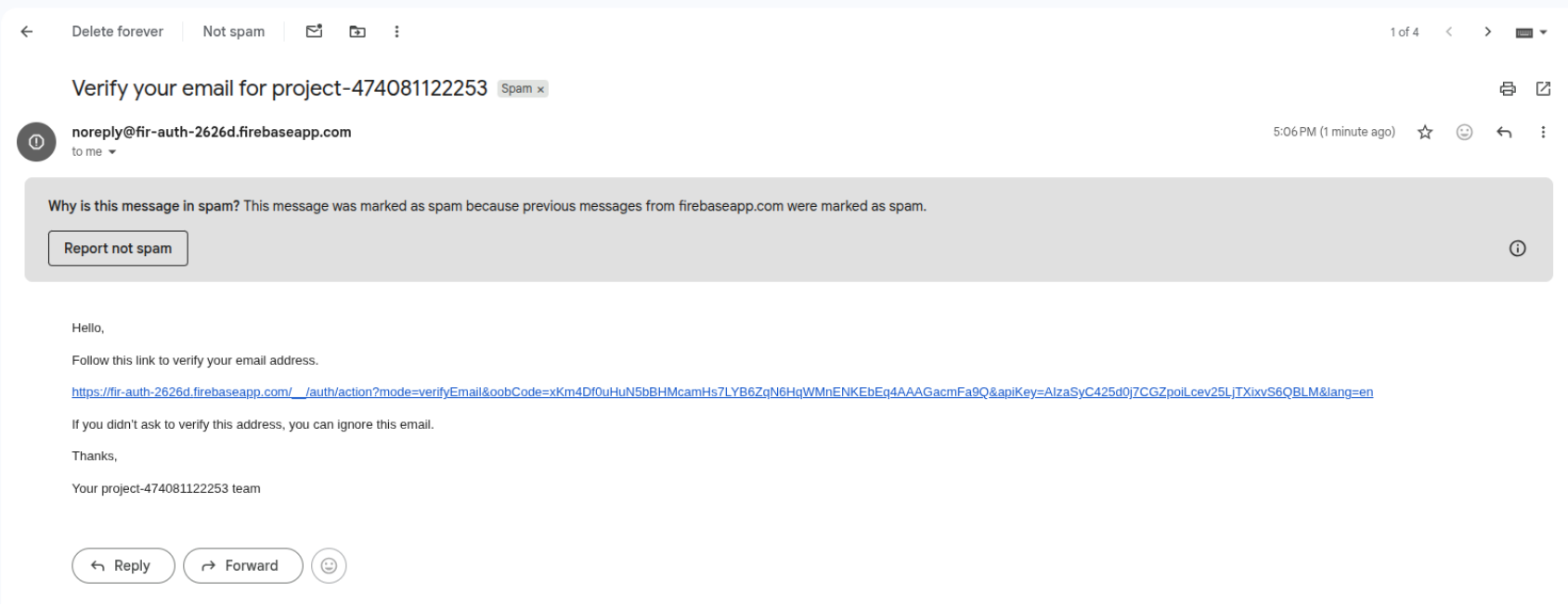
```
ElevatedButton.icon(
  onPressed: () async {
    await user.reload();
    final currentUser = FirebaseAuth.
     instance.currentUser;
    if (currentUser != null &&
     currentUser.emailVerified) {
      ScaffoldMessenger.of(context).
     showSnackBar(
        SnackBar(
          content: Text('Email da duoc
     xac thuc!'),
          backgroundColor: Colors.green,
        ),
      );
      setState(() {});
    } else { //.. send error
    }
  },
  icon: Icon(Icons.refresh),
  label: Text('Toi da xac thuc'),
  // ...
),
```

Xác minh Email (Con't)





Xác minh Email (Con't)



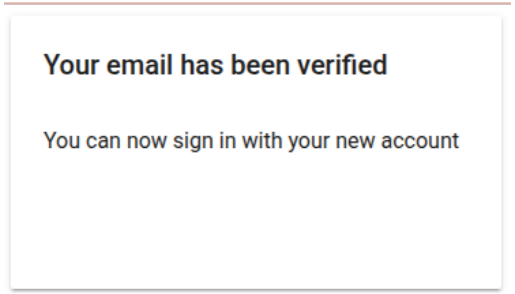


Table of Contents

1. Overview
2. Email/Password login
3. Social Login
   - Google Login
   - Facebook Login
4. User Management
5. Security
6. Tài liệu tham khảo

Security

- **Firebase Authentication** chỉ hỗ trợ việc đăng nhập và xác thực người dùng.
- Để bảo vệ dữ liệu an toàn, cần kết hợp với **Firestore Database** và thiết lập **Security Rules** nhằm:
  - Phân quyền giữa người dùng thường và quản trị viên
  - Đảm bảo xác thực an toàn qua email hoặc các phương thức khác
  - Quản lý trạng thái, cấp quyền hoặc xóa người dùng khi cần thiết

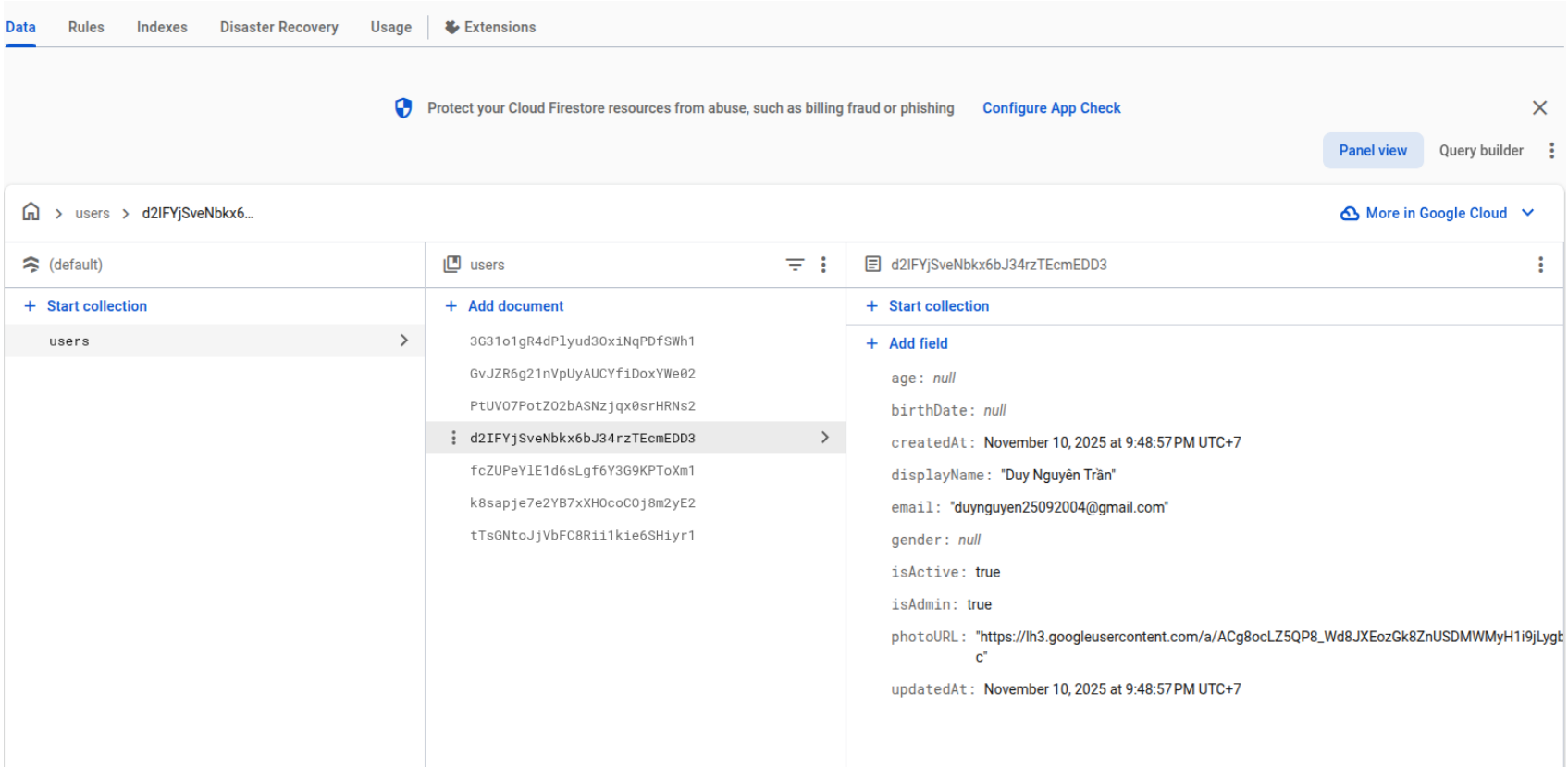
Phân quyền người dùng trong Firestore

- **Mục tiêu:** Chỉ cho phép người có quyền truy cập hoặc chỉnh sửa dữ liệu phù hợp.
- **Cách cấu hình:** Vào *Firebase Console* → *Firestore Database* → *Rules* để thiết lập quy tắc cho cơ sở dữ liệu.

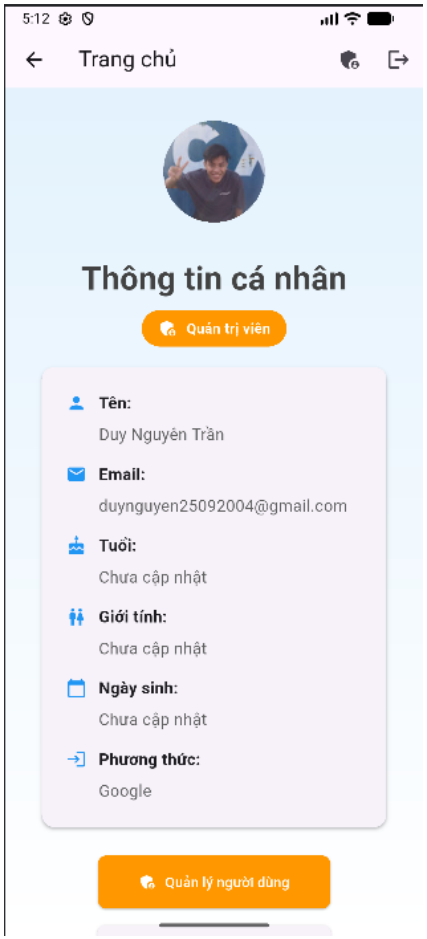
```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    function isAdmin() {
      return request.auth != null &&
             get(/databases/$(database)/documents/
     users/$(request.auth.uid))
             .data.isAdmin == true;
    }
    function isSignedIn() {
      return request.auth != null;
    }
    function isOwner(userId) {
      return request.auth != null && request.auth.
     uid == userId;
    }
    match /users/{userId} {
      allow read: if isOwner(userId) || isAdmin();
      allow create: if isSignedIn() && isOwner(
     userId);
      allow update: if isOwner(userId) || isAdmin();
      allow delete: if isAdmin();
    }
  }
}
```

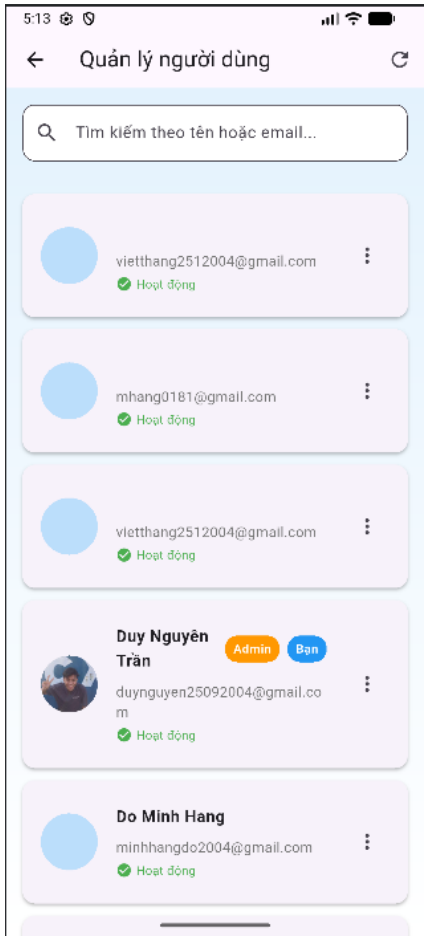
Phân quyền người dùng trong Firestore (Con't)

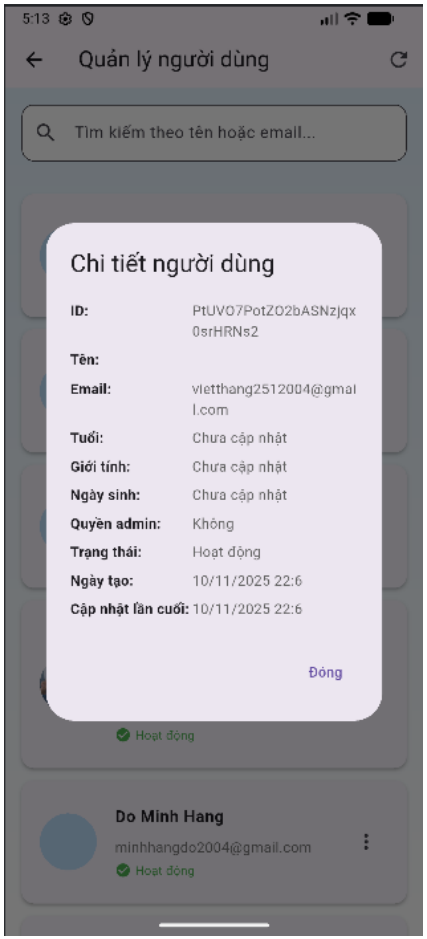
- Sau khi thiết lập các Rules trong database xonng, ta sang mục Data để thay đổi quyền của người dùng.



Demo phân quyền người dùng







Vô hiệu hóa và xóa tài khoản

- Để xóa 1 dữ liệu trên Firebase, ta sử dụng phương thức: **_firestore.collection('users').doc(userId).delete()**.

```
Future<void> _deleteUser(String userId, String email) async {
  final confirm = await showDialog<bool>(
    context: context,
    builder: (context) => AlertDialog(
      title: Text('Xac nhan xoa'),
      content: Text(
        'Ban co chac chua?.',
      ),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(context, false),
          child: Text('Huy'),
        ),
```

```
        ElevatedButton(
          onPressed: () => Navigator.pop(context, true),
          style: ElevatedButton.styleFrom(backgroundColor: Colors.red),
          child: Text('Xoa'),
        ),
      ],
    ),
  );
  if (confirm != true) return;
  try {
    await _firestore.collection('users').doc(userId).delete();
    _showSuccess('Da xoa');
  } catch (e) {
    _showError('Loi: ${e.toString()}');
  }
}
```

Security Best Practices

- **Xác thực người dùng:** Kiểm tra xác thực bằng email cho các hoạt động nhạy cảm (ví dụ: khôi phục mật khẩu) để tránh tạo tài khoản giả hoặc truy cập trái phép.
- **Phân quyền truy cập:** Thiết lập các *security rules* trong Firestore Database để giới hạn quyền truy cập và chỉnh sửa dữ liệu phù hợp với từng loại người dùng.
- **Quản lý tài khoản:** Quyền *xóa tài khoản* chỉ nên được cấp cho **admin** nhằm đảm bảo tính toàn vẹn và an toàn của dữ liệu.

Table of Contents

1. Overview
2. Email/Password login
3. Social Login
   - Google Login
   - Facebook Login
4. User Management
5. Security
6. Tài liệu tham khảo

Tài liệu tham khảo

ThamKhao.png

THANK YOU FOR LISTENING